LLM-Augmented Hybrid Movie Recommender

Project Proposal - CSE 573: Semantic Web Mining Group 18

Team 18

Atharva Bhavin Thaker · Mawadda Abuhannoud · Pranjal Padakannaya · Rahma Abuhannoud · Sachin Shivanand Shankarikoppa · Sanjay Soralamavu Dev

Arizona State University

Copyright © 2023 Arizona Board of Regents

Problem Definition

Problem

- Modern streaming platforms contain **thousands of movies**, making it difficult for users to discover relevant content
- Users often spend significant time browsing but still fail to find movies that match their preferences
- There is a strong need for **personalized recommendation systems** that can guide users effectively

Challenges

- **Sparse interaction data**: Most users rate only a small subset of movies
- **Cold-start problem**: New users and new movies lack sufficient history
- **Dynamic preferences**: User interests may change across sessions or moods
- **Natural language queries**: Users expect systems to understand queries like *"movies like Interstellar"*

Goal

- Develop a **hybrid recommendation system**
- Combine multiple models to capture different user signals
- Provide accurate, context-aware, and personalized recommendations
- Support natural language interaction

Copyright © 2023 Arizona Board of Regents

Dataset Description

MovieLens 20M × TMDB API

MovieLens 20M

Primary Behavioral Dataset

20M	138K	27K
Ratings	Users	Movies

KEY FILES

`ratings.csv`
userId · movieId · rating (0.5–5.0) · timestamp

`movies.csv`
movieId · title (with year) · pipe-separated genres

`links.csv`
movieId · imdbId · tmdbId ← bridge to TMDB API

Jan 1995 – Mar 2015 · All users rated ≥ 20 movies · Clean, no missing values

TMDB API

Contextual & Visual Metadata Layer

~24K
Movies with Full Metadata

Free for academic use · Requires API key
85–90% coverage of MovieLens catalog

KEY METADATA FIELDS (/movie/{id})

`title / overview`
Movie title & plot summary

`keywords`
Thematic tags → Knowledge Graph

`genres`
Genre objects with IDs & names

`release_date/runtime`
Release metadata

`cast / crew`
Top actors and director

`popularity/vote_avg`
Crowd-sourced quality signals

Bulk-fetched via tmdbId · Cached locally for reproducibility

Integration: MovieLens × TMDB joined on tmdbId → Unified dataset: behavioral signals + semantic metadata → Powers OCCF, GRU4Rec & Knowledge Graph

State-of-art Methods and Algorithms

Implicit Collaborative Filtering (OCCF / Matrix Factorization)

- Models implicit feedback (views, clicks, ratings ≥ 3) using confidence-weighted matrix factorization.
- Captures stable, long-term taste patterns across MovieLens 20M.
- Robust to noisy implicit signals and large-scale datasets.

What we implement:

- Train OCCF on MovieLens 20M to model stable user taste
- Use predicted interaction scores as the long-term preference component in final ranking

Session-Based Recommendation (GRU4Rec)

- Uses recurrent neural networks (GRU) to model sequential user behavior
- Predicts next-item probability based on recent interaction sequence
- Captures short-term intent and temporal dynamics within sessions

What we implement:

- Train GRU4Rec on user interaction sequences
- Generate session-aware scores reflecting current user intent
- Integrate as the short-term preference signal in hybrid model

Copyright © 2023 Arizona Board of Regents

State-of-art Methods and Algorithms

Knowledge Graph-Based Recommendation

- Encodes relationships among movies, genres, actors, directors, keywords.
- Enables multi-hop reasoning and cold-start support using metadata.
- Graph-based similarity improves diversity and semantic relevance

What we implement:

- Build Neo4j knowledge graph using TMDB metadata
- Compute similarity using graph traversal and entity relationships
- Use graph-based scores as context-aware component
- Enable explainability via graph paths (e.g., shared actor, genre)

LLM-Augmented Recommendation

- Interprets natural language queries into structured signals by extracting intent, constraints, and semantic meaning
- Re-ranks candidate movies beyond what interaction data alone can capture and generates human-readable explanations for every recommendation.

What we implement:

- Deploy a local LLM to parse natural language user queries
- Extract structured intent (genre, mood, seed movies, constraints) and map to TMDB metadata fields
- Re-rank top-K candidates from the hybrid model using query relevance and Knowledge Graph context
- Generate natural language explanations using KG paths (e.g., shared director, actor, or thematic keyword)
- Handle cold-start users by deriving initial preferences from query semantics rather than interaction history

Copyright © 2023 Arizona Board of Regents

Research and Development Plan

Phase 1 - Data Preparation

- Preprocess MovieLens 20M dataset and remove inconsistencies
- Integrate TMDB metadata using tmdbId (links.csv)
- Create chronological 80/10/10 train-validation-test split (no leakage)
- Construct user sessions using timestamp-based time-gap rules
- Build knowledge graph: Movie → Genre, Actor, Director, Keyword

Phase 2 - Core Model Development

- OCCF (Long-term Preference): Train confidence-weighted matrix factorization (ALS / LightFM); Learn latent user–item embeddings from implicit feedback; Outputs relevance scores representing stable user preferences
- GRU4Rec (Short-term Intent): Train GRU on session-based interaction sequences; Use ranking losses (TOP1 / BPR) for next-item prediction; Outputs session-aware scores capturing recent user intent
- Knowledge Graph (Semantic Context): Construct Neo4j graph from TMDB metadata; Generate embeddings using PyKEEN / DGL-KE; Compute semantic similarity scores via graph traversal

Phase 3 - Fusion, LLM integration & evaluation

- Combine model outputs using fusion layer
- Perform ablation studies to evaluate contribution of each component
- Analyze performance on cold-start users and items
- Prepare system for final demo and qualitative evaluation

Copyright © 2023 Arizona Board of Regents

Research and Development Plan

Fusion & Re-ranking Layer

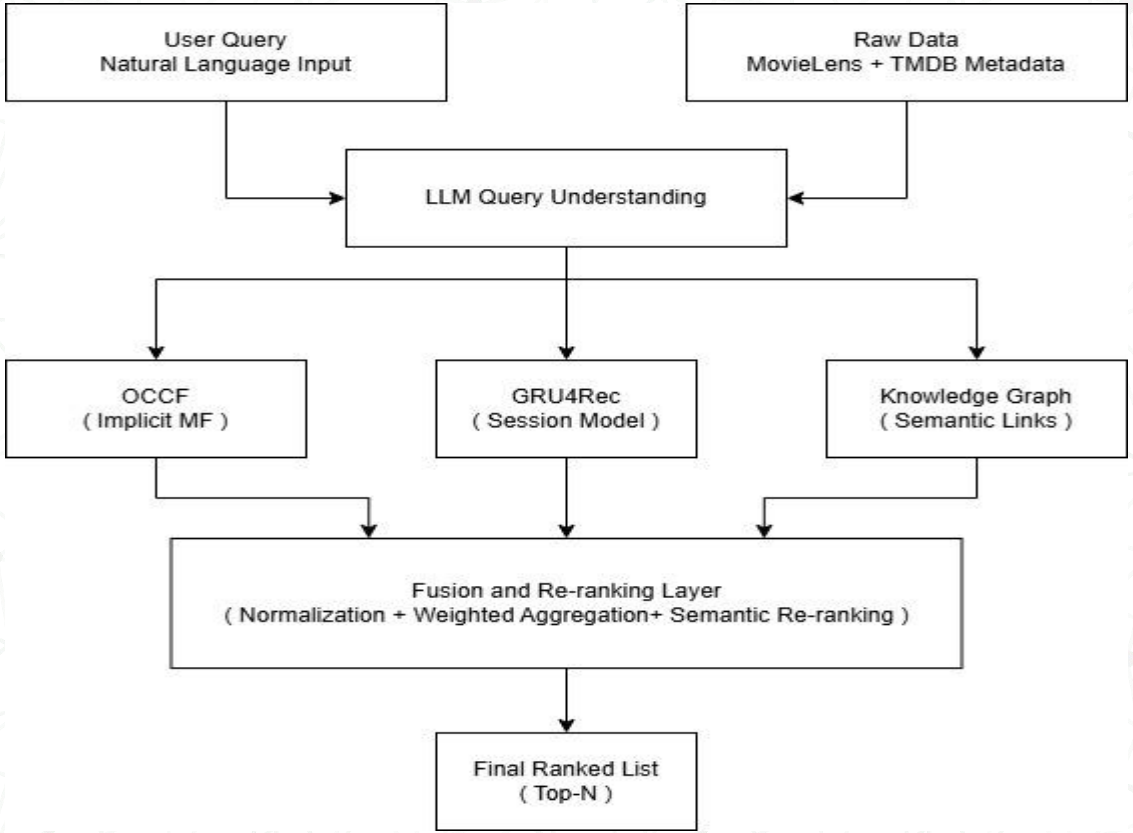
- Score Normalization: Apply min-max normalization to each model's output scores independently before combining. This ensures no single model dominates. OCCF scores, GRU4Rec probabilities, and KG scores must be brought to a common [0,1] scale before aggregation
- Weighted Aggregation: $FinalScore(u,i) = \alpha \cdot s_OCCF + \beta \cdot s_GRU + \gamma \cdot s_KG$. Weights initialized at (0.4, 0.3, 0.3) reflecting the primacy of long-term preference, then tuned via grid search over the validation set using NDCG@10 as the objective.
- LLM-based Re-ranking: When a natural language query is present, the top-K candidates from the fusion layer are passed to the LLM along with KG context. The LLM scores each candidate against the parsed query intent and re-orders the final list
- Ablation Study: Compare normalization strategies (min-max, z-score, rank-based) before fusion. Evaluate each model branch independently against the full hybrid to isolate each component's contribution.

Role of LLM in Pipeline

- Converts natural language input into structured intent. Example: "dark sci-fi like Inception" -> {genre: Sci-Fi/Thriller, mood: dark/cerebral, seed_movie: Inception}
- Candidate Retrieval Guidance: Extracted entities mapped to Neo4j KG nodes; 1–2 hop traversal surfaces semantically related candidates beyond what interaction data captures.
- Cold-start Handling: For users with fewer than 10 interactions, LLM derives a preference profile from query text alone
- Explanation Generation: Produces one-sentence justifications per recommendation using KG paths. Example: "Both directed by Christopher Nolan and share themes of time and identity."
- LLM Selection: Mistral 7B Instruct deployed locally: strong instruction-following, runs on a single GPU or high-RAM CPU, no external API dependency.

Copyright © 2023 Arizona Board of Regents

High Level Implementation Flow Diagram



Copyright © 2025 Arizona Board of Regents

Design of Experiment and Evaluation Plan

Experiment design

Data & split strategy

MovieLens 20M provides ratings and timestamps; TMDB adds genres, cast, crew and keywords.

Use both an 80/10/10 per-user split and a chronological holdout built from each user's most recent interactions.

Systems compared

Popularity baseline
Neighborhood collaborative filtering
OCCF only
GRU4Rec only
Knowledge-graph only
Final fused hybrid ranker

Session protocol

Build sessions from timestamped interactions with a time-gap rule.

Evaluate next-item ranking inside sessions so short-term intent is measured separately from long-term preference.

End-to-end experimental workflow

1. Unified dataset
Join MovieLens ratings with TMDB metadata through tmdbId.

2. Model branches
Train OCCF, GRU4Rec and KG-based retrieval in parallel.

3. Score fusion
Normalize branch scores and tune fusion weights on validation data.

4. Final evaluation
Report ranking, session, cold-start and robustness results.

Copyright © 2026 Arizona Board of Regents

Design of Experiment and Evaluation Plan

Evaluation protocol

Ranking quality

Precision@10 and Precision@20
Recall@10 and Recall@20
NDCG@10 and NDCG@20
MAP@K for cross-user ranked-list comparison

LLM Re-ranking Quality

Compare NDCG@10 before and after LLM re-ranking on a held-out natural language query set to isolate the query layer's contribution.

Session metrics

For GRU4Rec, report HitRate@K and MRR@K for next-item prediction.

This isolates how much session signals improve immediate recommendation accuracy.

Cold-start & sparsity

Break out users with very few training interactions and long-tail movies with low counts.

Measure whether the knowledge graph reduces the performance drop in these settings.

Robustness checks

1. **Sweep OCCF latent size, GRU4Rec session windows and knowledge-graph embedding depth.**
2. **Use fixed random seeds and average multiple runs to reduce variance.**
3. **Compare score-normalization schemes before fusion across different user groups.**

Success Criterion

The hybrid system is considered successful if it meaningfully outperforms the ALS-only baseline on NDCG@10, and if cold-start performance degrades less than it would under pure collaborative filtering, demonstrating that the KG and LLM components provide measurable value beyond interaction data alone.

Copyright © 2023 Arizona Board of Regents

Project Timeline

Phase	Task	Deadline
Planning	Define problem, finalize objectives, datasets, and overall system design	Feb. 25, 2026
Data Preparation	Clean MovieLens data, validate identifiers, integrate TMDB metadata, build unified dataset	Mar. 7, 2026
Baseline Development	Implement popularity and neighborhood-based baselines	Mar. 14, 2026
Model Development	Implement OCCF, GRU4Rec, and initial Knowledge Graph construction	Apr. 11, 2026
Milestone Review	Present preliminary system, baseline results, and initial model outputs	Apr. 15, 2026
Evaluation	Run Precision@K, Recall@K, NDCG@K experiments and tune fusion weights	Apr. 17, 2026
Refinement	Tune fusion weights, address cold-start cases, finalize pipeline	Apr. 20, 2026
Final Presentation	Prepare complete system, results, and presentation for demonstration	Apr. 29, 2026
Documentation	Complete full write-up and submit final project	May 4, 2026

Copyright © 2023 Arizona Board of Regents

Division of Work

Member	Responsibilities
Pranjal	Set up project repository and environment; implement MovieLens preprocessing; create train/validation/test splits; document data pipeline.
Atharva	Implement baselines (popularity, user/movie mean, neighborhood-based CF); run initial comparisons and sanity checks; assist with evaluation scripts.
Sanjay	Implement OCCF-based collaborative filtering; tune hyperparameters for long-term preference modeling; support integration with the fusion layer.
Sachin	Implement session-based GRU4Rec (session construction, training, and next-item prediction); evaluate session metrics; assist with hybrid fusion design.
Rahma	Implement TMDB enrichment pipeline (bulk fetch, caching, schema); construct the Neo4j Knowledge Graph; generate metadata-based similarity signals.
Mawadda	Develop evaluation scripts (Precision@K, Recall@K, NDCG@K, MAP@K, HitRate@K, MRR@K); implement score normalization and fusion layer; support final demo integration and results tables.
All Members	Contribute to sanity testing, reporting, documentation, and preparation of presentation slides; review each other's work for consistency.

Copyright © 2023 Arizona Board of Regents

References

[1] M. Li, B. Dias, I. Jarman, and P. J. Lisboa, “Addressing cold start in recommender systems: A semi-supervised co-training algorithm,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 1205–1214.
[2] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, “Session-based recommendations with recurrent neural networks,” in Proc. 4th Int. Conf. Learning Representations (ICLR), San Juan, Puerto Rico, 2016.
[3] R. Mazumder, T. Hastie, and R. Tibshirani, “Top-N recommender system via matrix completion,” in Proc. 30th AAAI Conf. Artificial Intelligence, Phoenix, AZ, USA, 2016, pp. 179–185.
[4] Ghosh, “Simple matrix factorization example on the MovieLens dataset using PySpark,” Medium, [Online]. Available:https://medium.com/@connectwithghosh/simple-matrix-factorizationexample-on-the-movielens-dataset-using-pyspark-9b7e3f567536. [Accessed: Feb. 20, 2026].
[5] H.Wang, F. Zhang, M. Zhang, and X. Xie, “RippleNet: Propagating User Preferences on the Knowledge Graph for Recommender Systems,” in Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM), 2018, pp. 417–426.

Copyright © 2023 Arizona Board of Regents

THANK YOU